Plan Work with GitHub Issues and MCP

Session 22 | Product and Delivery Teams | Intermediate

Conversation becomes durable work

Session 21 produced an approved decision brief.

Session 22 turns it into real GitHub issues.

This is the third step in **Delivery Decision Studio**.

MCP in one sentence

An MCP server gives Copilot named tools for reading or changing another system.

Learners use the tools. They do not build the server.

A plugin may bundle an MCP server with skills or other customizations. Verify the active GitHub MCP connection itself before reading or writing issues.

An issue is a contract for the next person

The issue should let a contributor decide whether they can start, what success looks like, and who will review it.

Issue element	What it prevents
Outcome	Work that repeats a vague request
Acceptance evidence	"Done" without a way to prove it
Non-goal	Scope growth during implementation
Dependency	Work that starts before a needed decision
Ownership gap	Silent assumptions about who decides

Tickets are not busywork when they reduce the handoff cost.

Parent and child issues answer different questions

The parent issue holds the outcome, shared constraints, and review path. Each child issue owns one independently reviewable result.

```
Parent: Improve request-status visibility
  ├─ Child: Define the status data contract
  ├─ Child: Implement the API response
  └─ Child: Add the client status view
```

If two child issues must edit the same behavior at the same time, sequence them or clarify the design first.

Writing through MCP is still an operation

Copilot can prepare the issue plan. A person approves the write only after seeing the target repository and expected result.

After the write, fetch the issue again. This fresh read proves what the system stored, rather than what the tool claimed it stored.

The Markdown plan is a temporary preview. **The created GitHub issues are authoritative.**

Read before writing

1. Confirm the repository.
2. Inspect existing labels and milestones.
3. Search for duplicate issues.
4. Read the approved brief.
5. Report permission or context uncertainty.

Existing work needs a disposition

Candidate	Decision
Exact duplicate	Reuse or close according to repository policy
Partial overlap	Extend the existing issue and create only the missing work
Conflict	Keep separate and record why
Unrelated	Leave unchanged

The lab includes open, closed, overlapping, and conflicting status issues.

Decompose by outcome

Create one parent issue and bounded child issues.

Each child needs:

- an independently reviewable result;
- acceptance evidence;
- a non-goal;
- dependencies;
- an owner or visible ownership gap.

Preview the complete write

Before approval, show:

- repository owner and name;
- issue count;
- every title and body;
- labels and milestone;
- relationships;
- fields left unset.

People approve the operation

Copilot proposes the action.

Record **Approve**, **Revise**, or **Reject** before the write.

No silent batch creation.

Generated confirmation is not verification

After the write:

1. start a fresh request;
2. retrieve the issues;
3. compare them with the approved plan;
4. correct one defect;
5. retrieve the corrected issue again.

Lab result

- Temporary reviewed issue preview
- Authoritative live issue set
- One parent issue
- Three to five child issues
- Fresh-read verification
- Parent-to-child links that Session 23 reads directly

Stop if tool review or verification is unavailable.